ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Exercício Prático — Linguagem C

Matrizes • Campo de Batalha (RPG Tático)

Item	Descrição
Tema	Matrizes bidimensionais em C
Pré-requisitos	Vetores, matrizes, laços aninhados, funções e condicionais
Nível	Difícil
Entrega	Arquivo-fonte .c compilando sem erros e sem avisos

1. Objetivo

Desenvolver o mapa de um jogo de estratégia por turnos utilizando uma matriz bidimensional de caracteres. O aluno deve praticar leitura e escrita em matrizes, percorrer os elementos com laços aninhados, validar limites de índices e organizar o programa em funções.

2. Descrição do problema

O tabuleiro do jogo é representado pela matriz char mapa[10][10]. Cada posição armazena um caractere que indica o que existe naquela casa do campo de batalha:

Símbolo	Significado
.	Terreno livre
#	Obstáculo (pedra, muro) — bloqueia o movimento
H	Herói — existe apenas um no mapa
I	Inimigo
$	Tesouro

No início da execução, o mapa deve estar totalmente preenchido com terreno livre. Em seguida, o usuário posiciona os elementos e movimenta o herói por meio de um menu.

3. Requisitos obrigatórios

O programa deve conter um menu em laço, repetido até que o usuário escolha sair. Cada funcionalidade abaixo deve ser implementada em uma função separada — não é permitido escrever toda a lógica dentro da main.

1. inicializarMapa() — preenche todo o tabuleiro com o caractere de terreno livre.
2. exibirMapa() — desenha o mapa no terminal, com a numeração das linhas e das colunas.
3. posicionarElemento() — coloca um símbolo na coordenada informada pelo usuário, recusando a operação se a posição já estiver ocupada ou se estiver fora do mapa.

4. moverHeroi() — recebe uma direção (W, A, S ou D) e move o herói uma casa. O movimento deve ser bloqueado se houver obstáculo ou se o herói sair do tabuleiro. Ao cair sobre um tesouro, exibir "Tesouro coletado!"; ao cair sobre um inimigo, exibir "Combate iniciado!".
5. contarInimigos() — retorna quantos inimigos ainda existem no mapa.
6. Menu principal com opção de encerrar o programa.

4. Função obrigatória

Implemente a função abaixo, respeitando exatamente esta assinatura:

```
int inimigosProximos(char mapa[10][10], int linha, int coluna);
```

A função deve retornar quantos inimigos existem nas 8 casas vizinhas à posição informada: acima, abaixo, à esquerda, à direita e nas quatro diagonais.

4.1 Regras

- A própria casa central não entra na contagem.
- Se a posição estiver na borda ou no canto do mapa, as casas inexistentes simplesmente não são contadas — o programa não pode, em hipótese alguma, acessar índices inválidos.
- A função deve ser chamada após cada movimento do herói para exibir um alerta na tela.

4.2 Alertas exibidos após o movimento

Retorno da função	Mensagem exibida
0	Área segura.
1 ou 2	Cuidado, inimigos por perto!
3 ou mais	PERIGO! Você está cercado!

5. Exemplo de saída esperada

```
     0  1  2  3  4  5  6  7  8  9
  0  .  .  #  .  .  .  .  .  .  .
  1  .  I  .  .  $  .  .  .  .  .
  2  .  .  H  .  .  .  #  .  .  .
  3  .  I  .  I  .  .  .  .  .  .

>> PERIGO! Voce esta cercado! (3 inimigos ao redor)
```

6. Pontos de atenção

- Verifique os limites do mapa antes de ler cada vizinho; caso contrário o programa acessa memória inválida e pode ser encerrado pelo sistema.
- Evite escrever oito blocos if repetidos: a varredura das 8 direções pode ser resolvida com dois laços aninhados variando de -1 a 1, ignorando a posição (0, 0).
- Ao mover o herói, mantenha a matriz coerente: limpe a casa antiga com o caractere de terreno livre antes de escrever na casa nova.
- Todas as funções devem receber a matriz como parâmetro; não utilize variáveis globais.

7. Critérios de avaliação

Critério	Peso
Funcionamento correto de inimigosProximos(), inclusive nas bordas	30%
Movimentação do herói com validação de limites e obstáculos	25%
Organização em funções e ausência de variáveis globais	20%
Exibição e formatação do mapa no terminal	15%
Validação das entradas do usuário e clareza do código	10%

Bom trabalho!